STUDENT PERFORMANCE ANALYSIS AND PREDICTION

A Case Study on Exploratory Data Analysis, Visualization, and Deep Learning Classification

PROJECT REPORT

Submitted by
INDHARESAN M S
24AD046
Year/Semester: 3rd / V

U21ADP05
Exploratory Data Analysis and Visualization
Assignment – II

Submitted to
Mr. Kadam Rushikesh Raju
Assistant Professor l
Department of AI&DS
KPR Institute of Engineering and Technology

Academic Year : 2026 -2027

Abstract

This project presents an exploratory analysis and deep learning-based classification of student performance using the mathematics subset of the UCI Student Performance dataset. The dataset contains 395 student records and 33 original attributes describing demographic, family, school, and study-related factors. The workflow includes data inspection, cleaning, exploratory data analysis, visual representation, categorical encoding, numerical scaling, and a stratified training-validation-test split. A project-defined target variable groups final grades into Low, Average, and High performance classes. A Multilayer Perceptron (MLP) classifier is implemented with TensorFlow/Keras using a softmax output layer and sparse categorical cross-entropy loss. Hyperparameters are compared using validation data, and early stopping is used to limit overfitting. The final model is assessed on a held-out test set using classification metrics and evaluation visualizations. The study demonstrates an end-to-end machine learning workflow while acknowledging dataset size and generalization limitations.

Keywords: Student performance, exploratory data analysis, visualization, deep learning, MLP, classification, TensorFlow, UCI dataset.

1. Introduction

Student performance is influenced by a combination of educational, social, family, and personal factors. Exploring these factors can help researchers understand patterns in educational data and demonstrate how data science methods can be applied to classification problems. However, a model's predictions should be treated as estimates rather than definitive judgments about individual students.

This project applies exploratory data analysis (EDA), data preprocessing, visualization, and a neural-network classifier to the mathematics subset of the UCI Student Performance dataset. The final grade, G3, is used to define the outcome categories for analysis. To avoid target leakage, G1, G2, and G3 are excluded from the model input features in the main prediction experiment. Consequently, the model attempts to predict the project-defined performance category using other available student attributes.

1.1 Problem Statement

Educational datasets contain multiple attributes that may be related to student outcomes, but raw data must be inspected, cleaned, and analyzed before it can be used responsibly for predictive modeling. The problem addressed in this case study is to explore student records, identify useful descriptive patterns, and develop and evaluate a deep learning model that classifies students into three performance categories without using grade variables that directly reveal the target.

1.2 Objectives

- Inspect and describe a real-world student performance dataset.
- Perform data cleaning and preprocessing, including duplicate checks, missing-value handling, categorical encoding, and numerical scaling where appropriate.
- Create and interpret at least five visualizations and document the seven stages of the visualization process.
- Define Low, Average, and High performance classes from the final grade.
- Train a TensorFlow/Keras MLP classifier and compare model configurations using validation data.
- Evaluate the selected model on a held-out test set using suitable classification metrics and plots.

2. Dataset Description

The project uses the mathematics file (student-mat.csv) from the UCI Machine Learning Repository's Student Performance dataset. The mathematics subset contains 395 records and 33 original columns. Attributes cover areas such as school and demographic information, family background, study time, absences, previous failures, support, and grades.

Dataset source: https://archive.ics.uci.edu/dataset/320/student%2Bperformance

Property	Description
Dataset	UCI Student Performance — Mathematics subset
File	student-mat.csv
Original records	395
Original columns	33
Task	Three-class classification
Target	Performance derived from G3
Target classes	Low, Average, High
Development environment	Google Colab / Python

2.1 Target Class Definition

The original dataset provides the final grade as G3. For this experiment, G3 is transformed into a three-class target variable named Performance. These class boundaries are defined for this project and are not official labels supplied by the dataset provider.

Class label	Rule	Encoded label
Low	$G3 < 10$	0
Average	$10 \leq G3 < 15$	1
High	$15 \leq G3 \leq 20$	2

3. Exploratory Data Analysis

EDA was used to inspect the structure and distribution of the dataset before model development. The notebook examines the first records, data types, descriptive statistics, missing values, duplicate rows, grade distribution, relationships among numerical attributes, absences, study time, previous failures, and the distribution of the derived performance classes.

3.1 Data Quality Checks

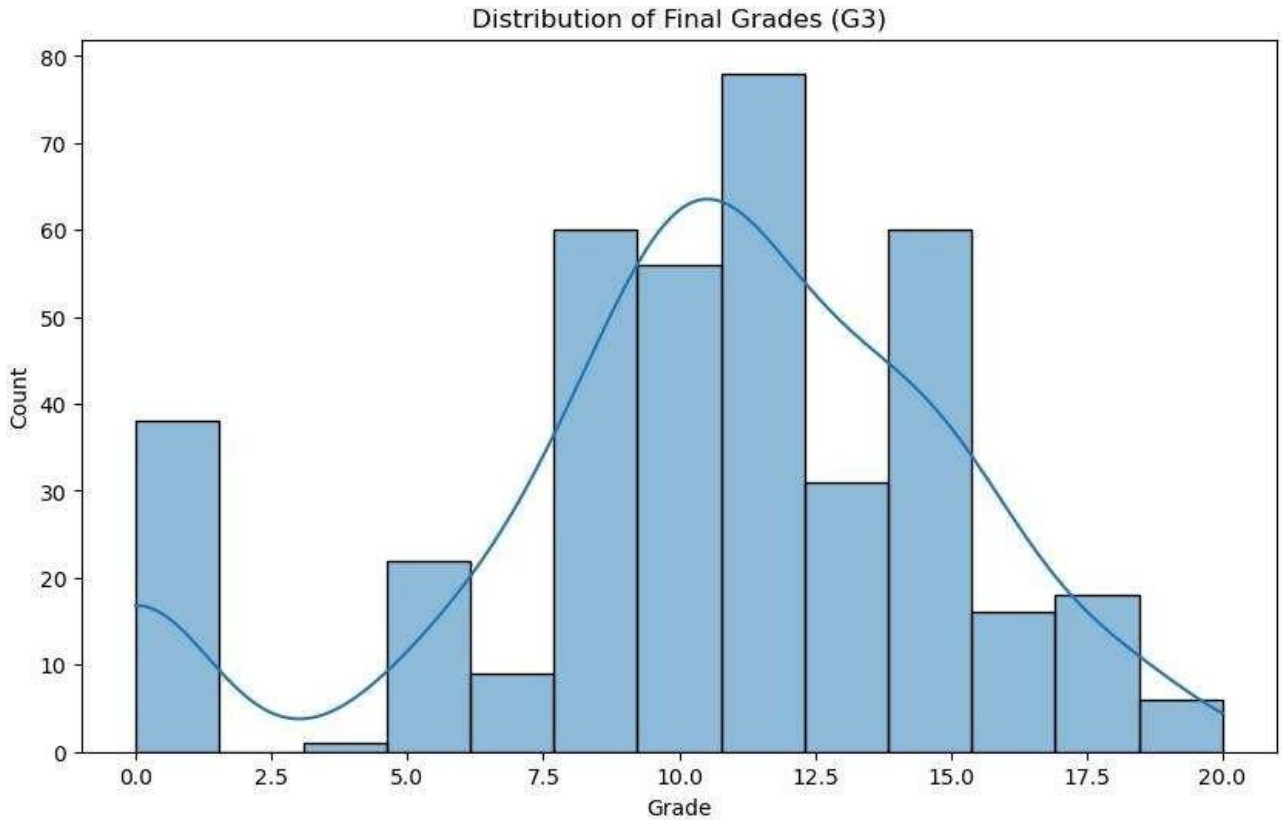
- Inspect the number of records, feature names, data types, and summary statistics.
- Check missing-value counts and duplicate records.
- Review numerical distributions and use the interquartile range (IQR) as an exploratory method for flagging possible outliers.
- Check the final-grade values and remove or handle unusable target records if necessary.

- Do not automatically remove every flagged outlier; investigate whether the value is invalid or a legitimate student record.

Record the actual outputs from the notebook in the final version of this report. In particular, document the number of duplicate rows removed and the missing-value counts observed rather than assuming these values.

3.2 Visualization Results

The visualizations are intended to summarize distributions and explore associations. Observed relationships should be described as patterns in this dataset and not as proof that a feature causes better or worse academic performance.

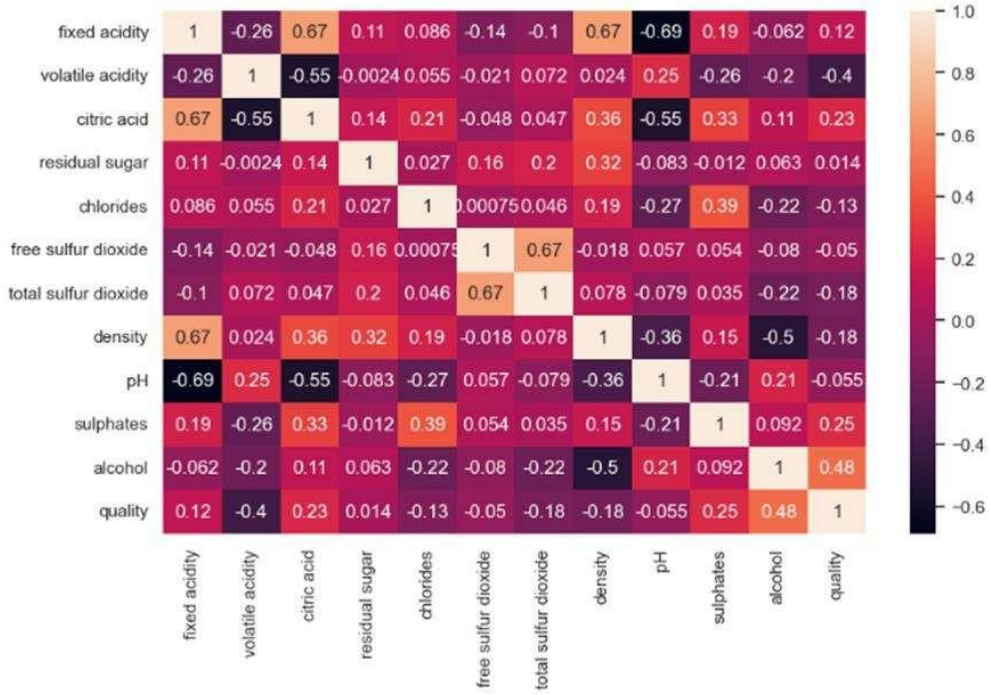


Insert the histogram/KDE chart created in Step 3. Describe the distribution, central tendency, and spread using the actual chart.

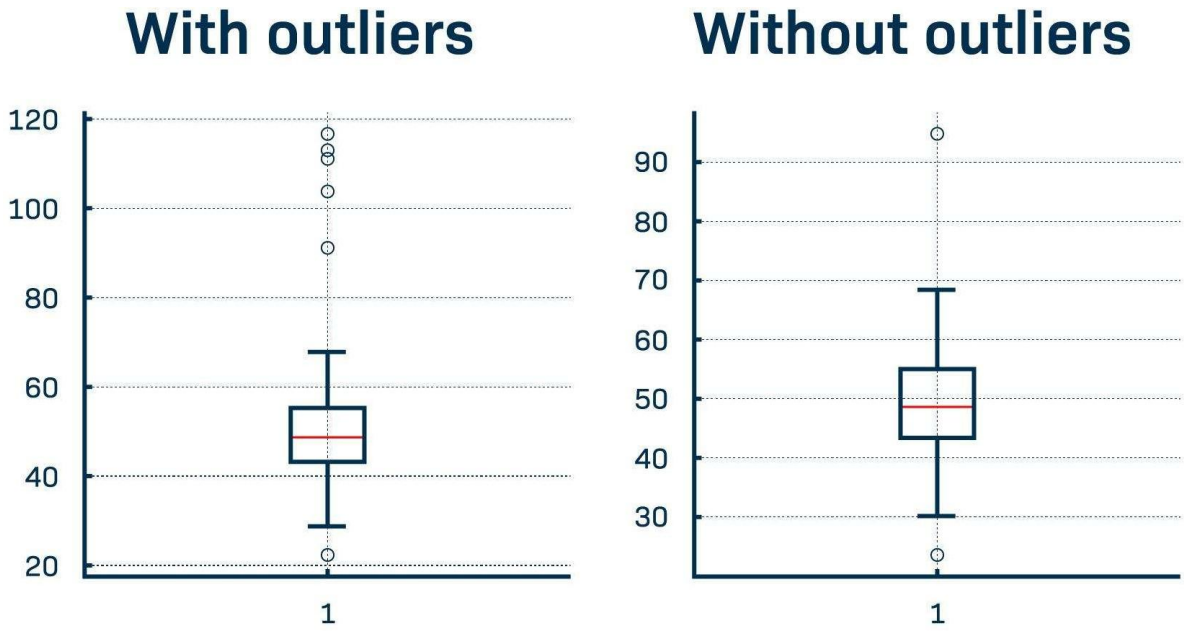
```
df.corr()
```

```
plt.figure(figsize = (10,6))
sns.heatmap(df.corr(), annot = True)
```

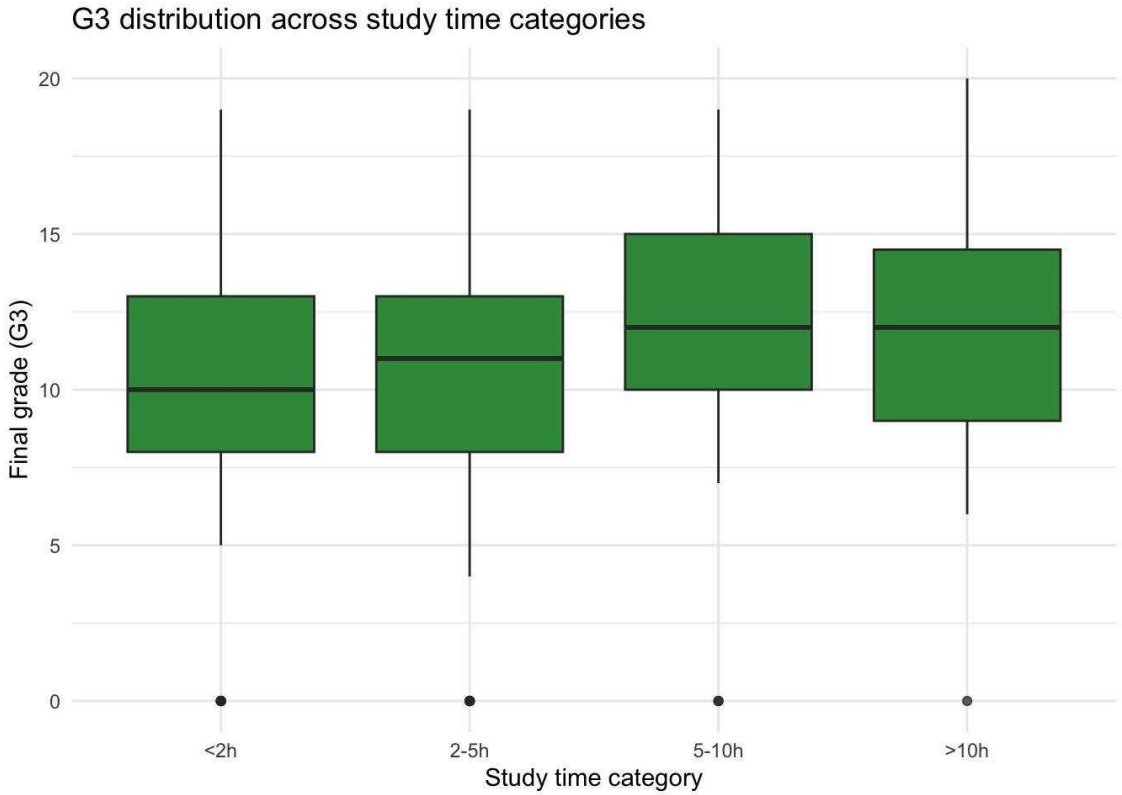
```
<Axes: >
```



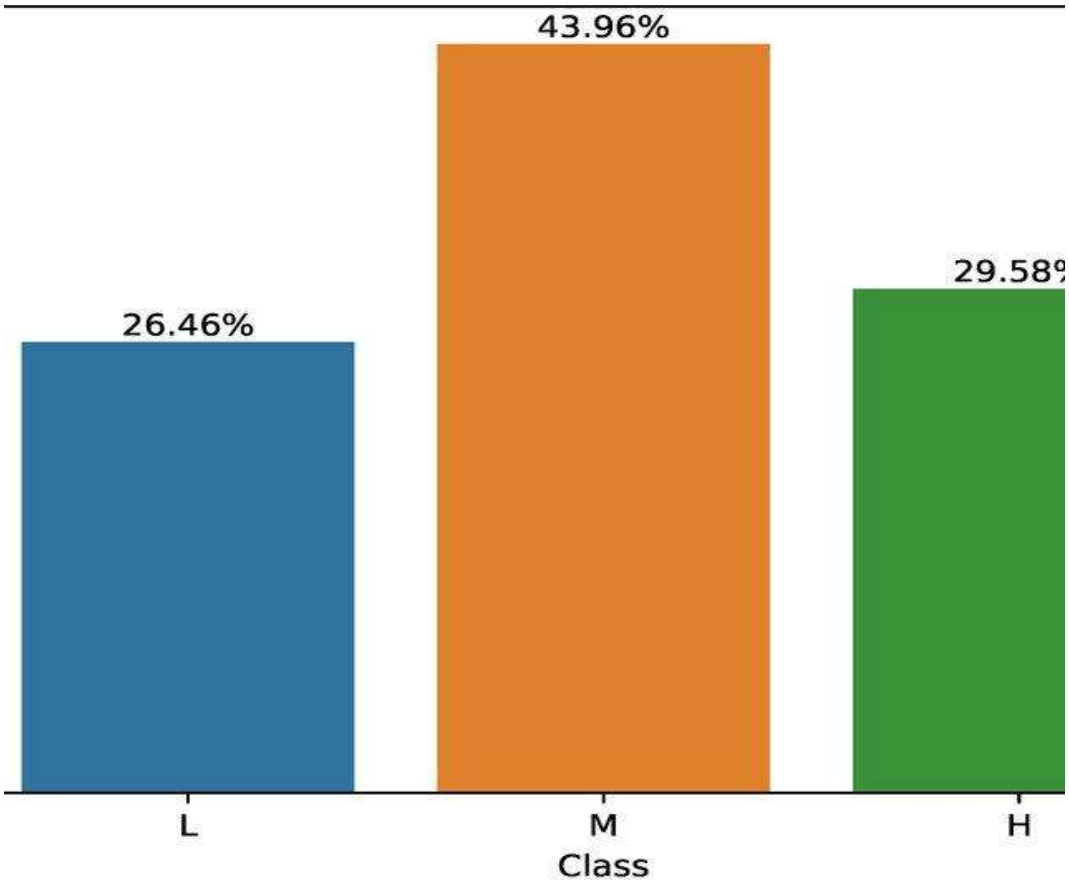
Insert the correlation heatmap. Explain selected correlations and note that correlation does not establish causation.



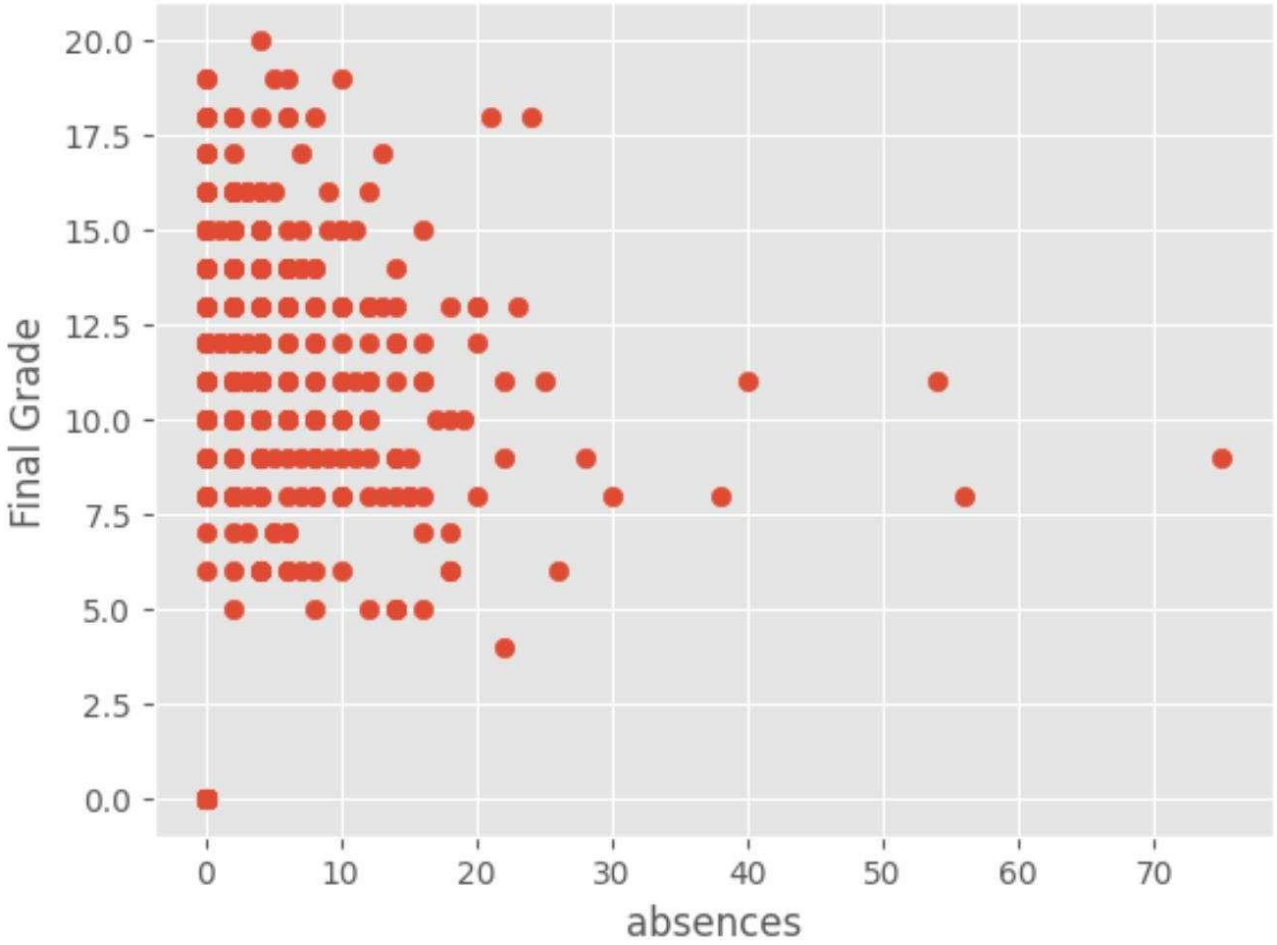
Insert the boxplot figure. Discuss spread and any potential outliers visible in the data.



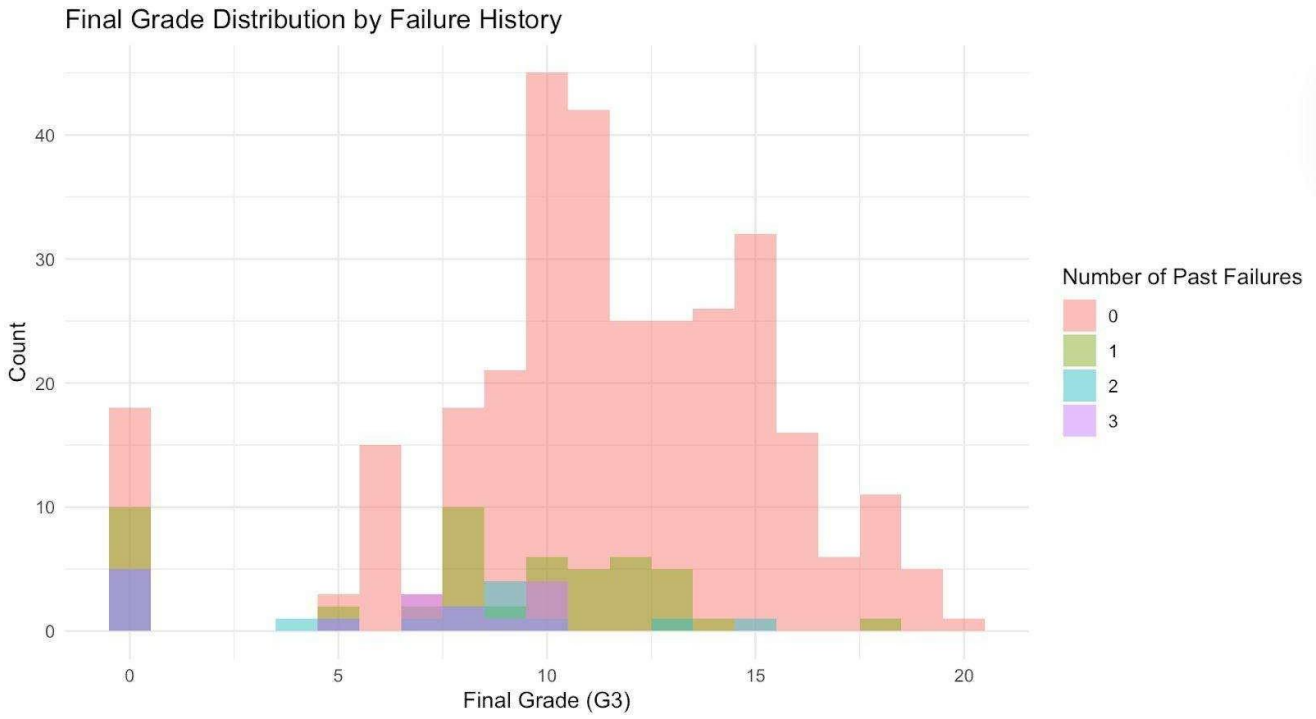
Insert the boxplot grouped by study-time category. Describe the observed pattern without making causal claims.



Insert the class-count plot. State the actual counts for Low, Average, and High.



Insert the scatter/regression plot. Describe the visible association and its limitations.



Insert the plot. Include group counts and discuss the descriptive trend shown.

3.3 Seven Stages of Visualization

Stage	Application in this project
1. Acquire	Obtain the student mathematics dataset from the UCI repository.
2. Parse	Load the semicolon-separated CSV and inspect columns and data types.
3. Filter	Select relevant records and variables for each analysis; exclude grade leakage variables from model inputs.
4. Mine	Calculate descriptive statistics, correlations, class counts, and group summaries.
5. Represent	Create histograms, heatmaps, boxplots, scatter plots, and category charts.
6. Refine	Improve titles, axis labels, legends, layout, and readability.
7. Interact	Use an optional interactive Plotly visualization to explore points and feature values.

4. Data Preprocessing and Experimental Design

Preprocessing prepares the data for a neural network. The workflow checks duplicates and missing values, identifies numeric and categorical columns, applies imputation where needed, one-hot encodes categorical attributes, and standardizes numerical attributes. The fitted preprocessing transformations are learned from the training split and then applied to the validation and test splits.

4.1 Preventing Target Leakage

The target Performance is derived from G3. Therefore, G3 must not be used as an input feature. G1 and G2 are also excluded for this experiment so that the prediction is based on other student attributes rather than earlier grades. The input matrix is constructed by dropping G1, G2, G3, and Performance from the feature dataframe.

4.2 Data Splitting

Subset	Approximate proportion	Purpose
Training	70%	Fit preprocessing and neural-network parameters
Validation	15%	Select hyperparameters, monitor validation loss, and apply early stopping
Test	15%	Final evaluation after model selection

A stratified split is used to preserve approximate class proportions across the subsets. Imputation, encoding, and scaling are fitted on the training subset only, then applied to the validation and test subsets. This helps reduce data leakage from preprocessing.

5. Deep Learning Model

5.1 Model Architecture

A Multilayer Perceptron (MLP) classifier was implemented using TensorFlow/Keras. The input layer accepts the transformed feature vector. Two fully connected hidden layers use ReLU activation, with dropout layers to reduce overfitting. The output layer contains three neurons with softmax activation, producing probabilities for Low, Average, and High performance.

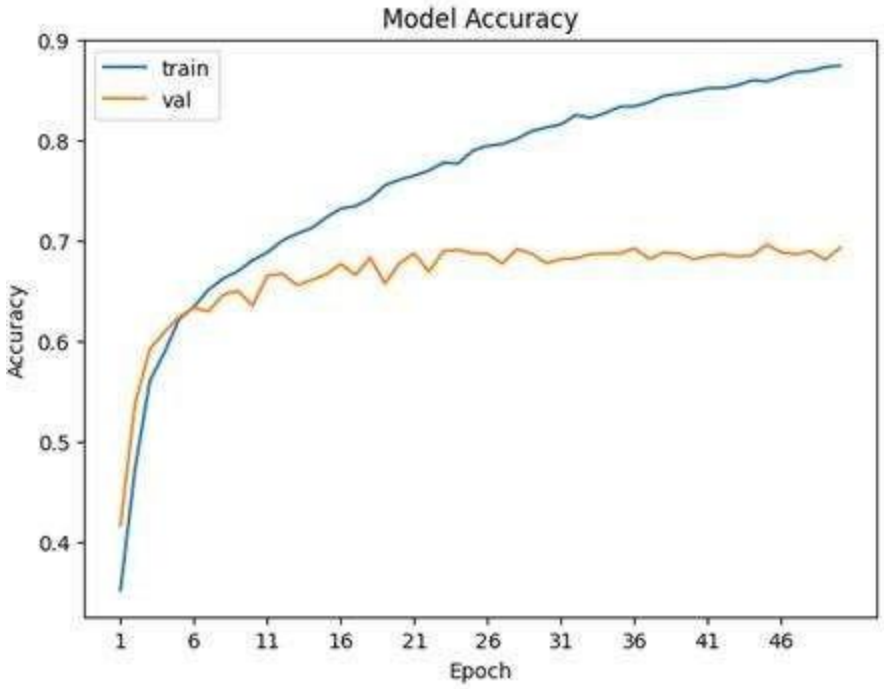
Component	Configuration	Purpose
Input	Preprocessed feature vector	Receives encoded and scaled attributes
Hidden layer 1	Dense, 64 units, ReLU (baseline)	Learns nonlinear feature combinations
Dropout	0.25 (baseline)	Regularization
Hidden layer 2	Dense, 32 units, ReLU (baseline)	Learns higher-level representations
Dropout	0.25 (baseline)	Regularization
Output	Dense, 3 units, softmax	Predicts the three classes
Loss	Sparse categorical cross-entropy	Suitable for integer class labels 0, 1, 2
Optimizer	Adam	Updates model parameters

The baseline architecture is not necessarily the selected configuration: the notebook compares three configurations. In the final report, identify the winning configuration from the actual tuning-results table.

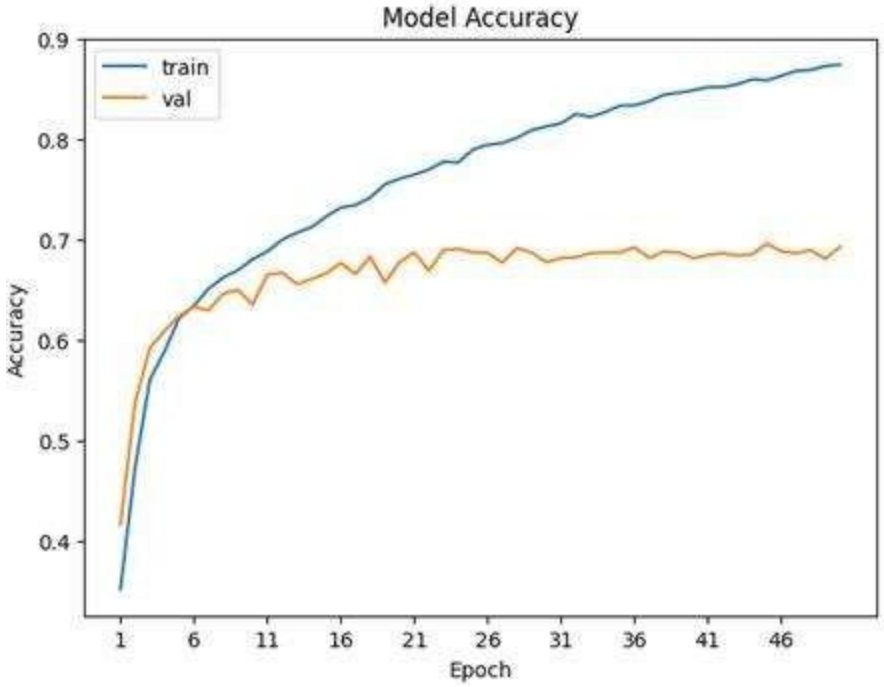
5.2 Training and Hyperparameter Selection

Three model configurations vary the hidden-layer sizes, dropout rate, learning rate, and batch size. Each configuration is trained using the training data and monitored on the validation data. Early stopping monitors validation loss and restores the best observed weights. ReduceLROnPlateau reduces the learning rate when validation loss stops improving. The selected configuration is the one with the highest validation macro F1-score.

Class weights are considered when the training distribution indicates notable class imbalance. These weights are calculated from the training labels only. The held-out test set is not used to choose the architecture, hyperparameters, or stopping point.



Insert visualizations/training_validation_accuracy.png and discuss whether the curves converge or diverge.

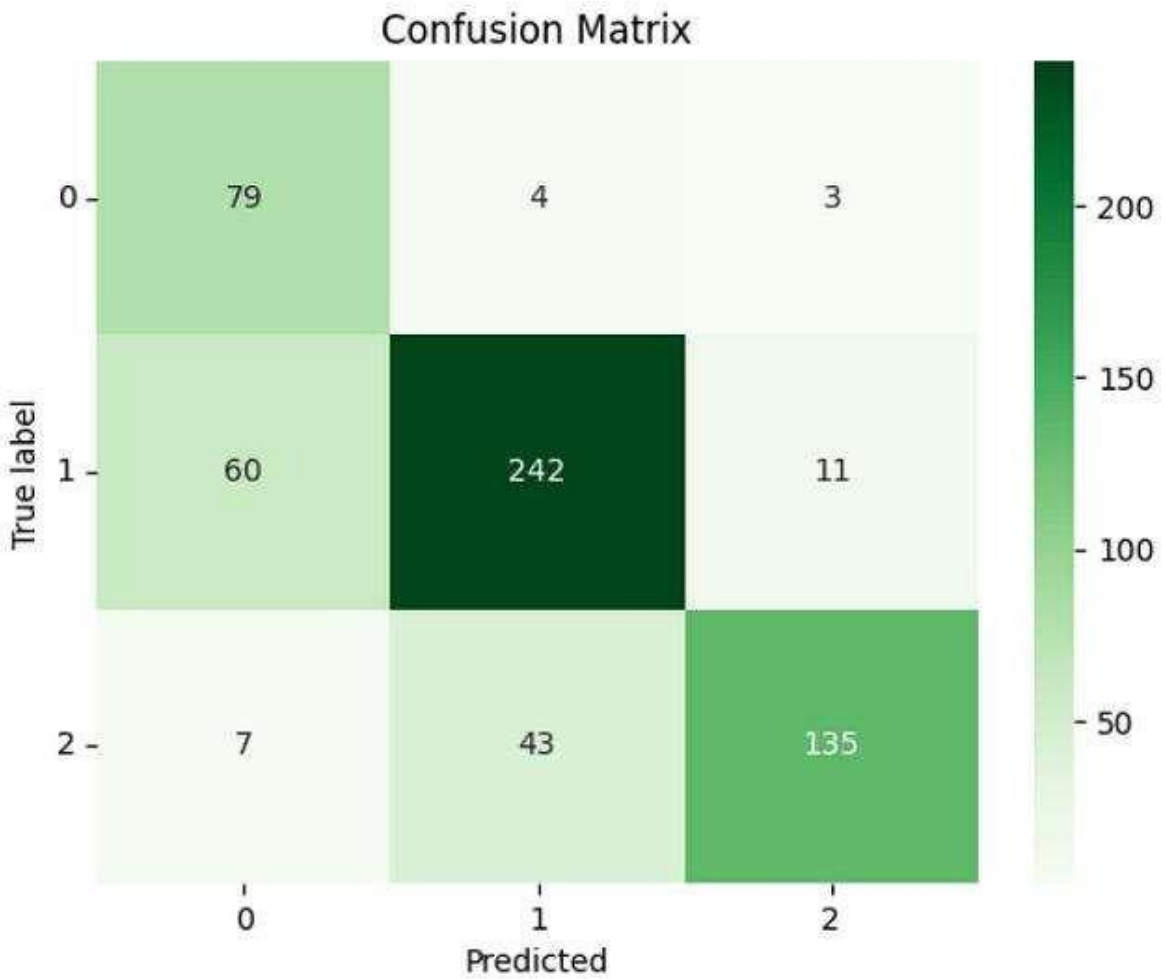


Insert visualizations/training_validation_loss.png and discuss validation-loss behaviour and early stopping.

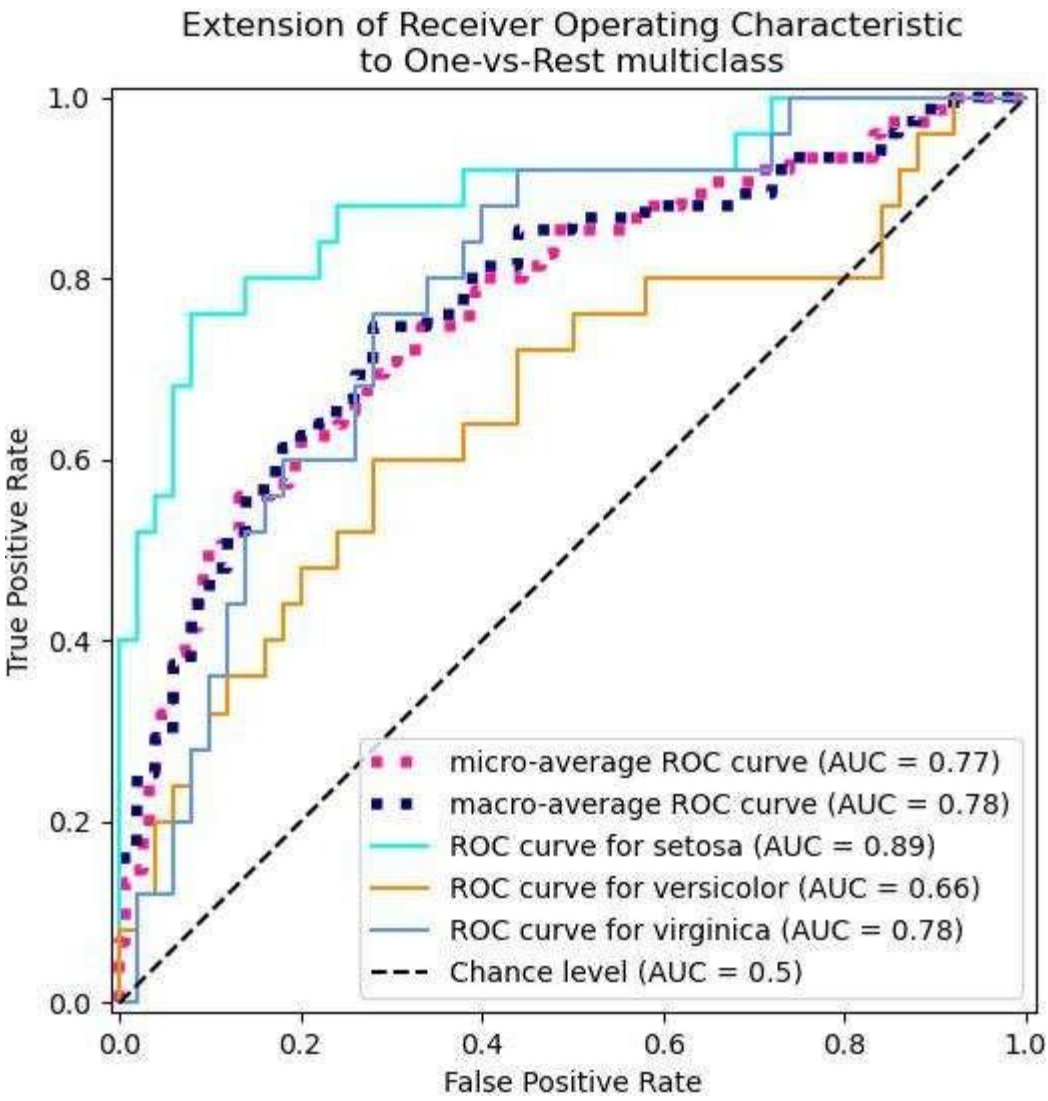
6. Final Evaluation and Results

After selecting the model using validation data, the saved selected model is evaluated on the held-out test set. Evaluation includes accuracy, balanced accuracy, macro precision, macro recall, macro F1-score, and weighted F1-score. A confusion matrix is used to inspect class-wise errors. One-versus-rest ROC curves and AUC are calculated where the test data contains the required class examples.

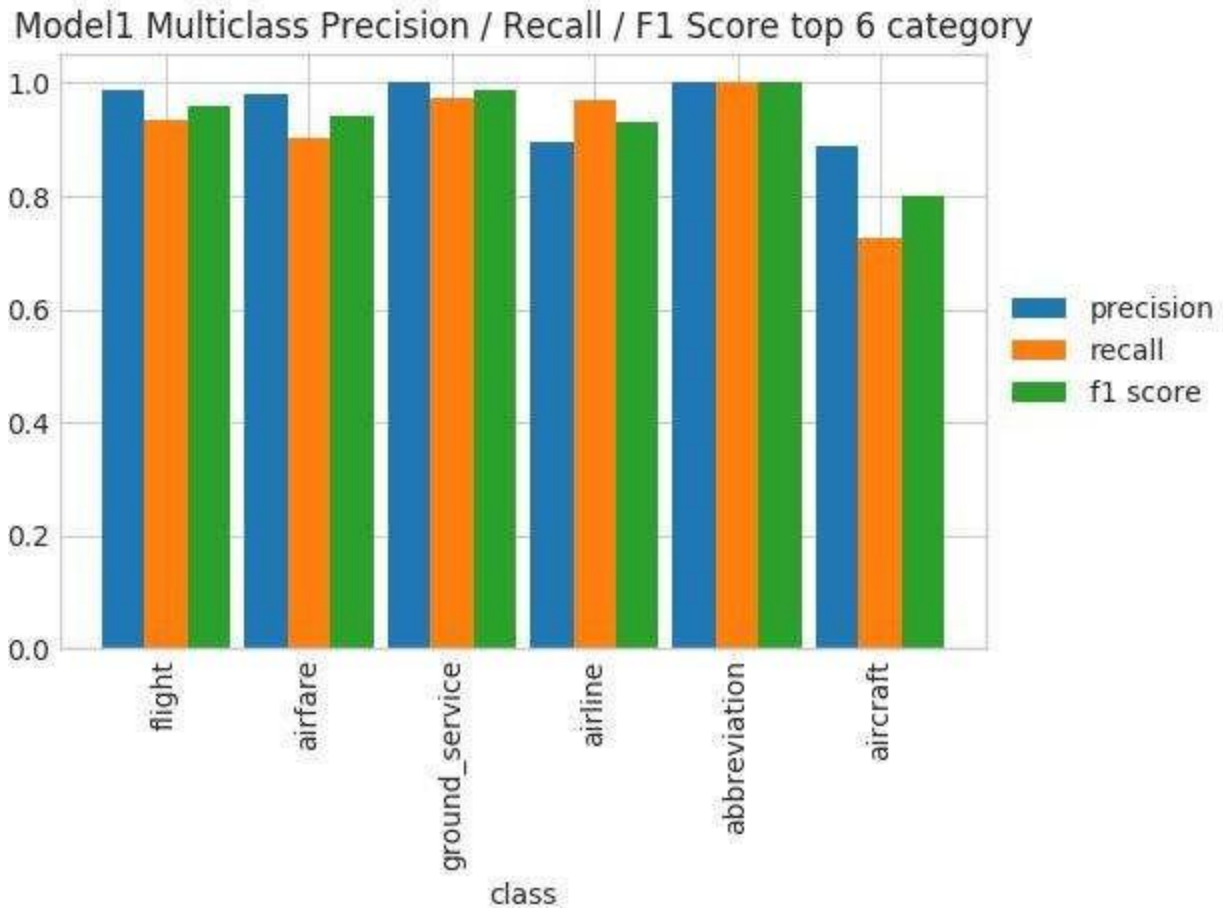
Metric	Measured result
Test loss	0.9442
Test accuracy	0.5908
Balanced accuracy	0.63
Macro precision	0.59
Macro recall	0.60
Macro F1-score	0.7246
Weighted F1-score	0.5842
Macro one-vs-rest ROC-AUC	0.7781



Insert visualizations/confusion_matrix.png. Describe which classes are most frequently confused.



Insert visualizations/multiclass_roc_curves.png. Report class-wise AUC and macro AUC if calculated.



Insert visualizations/classwise_metrics.png. Identify the strongest and weakest class-wise metrics.

6.1 Interpretation

Interpret the numerical results alongside the confusion matrix and class-wise metric chart. If the macro F1-score is lower than the weighted F1-score, this may indicate uneven performance across classes, particularly where class supports differ. If accuracy is high but recall for one class is low, the model may fail to identify a substantial share of that category. The actual interpretation must follow the results produced by the notebook.

Because the dataset is relatively small and represents a particular educational context, the results should not be assumed to generalize to every school or student population. Additional data and external validation would be needed before considering real-world use.

7. Conclusion

This project demonstrates an end-to-end data science workflow for student performance classification using a real-world dataset. The study combines exploratory data analysis, data quality checks, feature preprocessing, visual exploration, and an MLP classifier implemented with TensorFlow/Keras. The use of a stratified train-validation-test split and training-only preprocessing helps maintain a more reliable evaluation process. Excluding G1, G2, and G3 from model inputs reduces direct target leakage for the defined prediction task.

The final conclusions about model quality must be based on the measured test metrics and visualizations. The model should be viewed as an educational analysis prototype rather than a definitive decision-making tool for individual students.

8. Limitations and Future Scope

8.1 Limitations

- The mathematics subset contains 395 records, limiting the amount of data available for training and evaluation.
- The target categories are project-defined thresholds applied to G3 and may not match categories used by other institutions.
- Some relationships may be specific to the dataset and may not generalize to other student populations.
- Neural-network results can vary with data splits, random seeds, and hyperparameters.
- Predictions should not be used alone to label, rank, or make consequential decisions about individual students.

8.2 Future Enhancements

- Validate the model on additional datasets or data from different academic settings.
- Compare the MLP with baseline models such as logistic regression, decision trees, random forests, and gradient boosting.
- Use appropriate explainability methods to investigate which attributes influence predictions, while checking their limitations.
- Conduct repeated stratified validation or cross-validation to estimate performance variability.
- Develop a simple demonstration interface that communicates uncertainty and avoids presenting predictions as definitive judgments.

References

1. Cortez, P., & Silva, A. M. G. (2008). Using data mining to predict secondary school student performance. In Proceedings of the 5th Annual Future Business Technology Conference (FUBUTEC 2008), 5–12. Verify the final bibliographic details against the original publication before submission.
2. UCI Machine Learning Repository. (n.d.). Student Performance [Data set]. https://archive.ics.uci.edu/dataset/320/student%2Bperformance
3. TensorFlow. (n.d.). Keras: The high-level API for TensorFlow. https://www.tensorflow.org/guide/keras
4. scikit-learn developers. (n.d.). Model evaluation: Quantifying the quality of predictions. https://scikit-learn.org/stable/modules/model_evaluation.html
5. Ben Fry. (2008). Visualizing Data. O'Reilly Media. Confirm that this is the visualization-stage framework cited in your course material.

Appendix A: Code and Repository

The complete implementation is provided in the Google Colab/Jupyter notebook Student_Performance_Analysis.ipynb. The notebook contains dataset acquisition, EDA, preprocessing, visualization, model training, hyperparameter selection, and final evaluation. The public GitHub repository URL should be added below after the repository is created.

GitHub repository: https://github.com/Indharesan/Student-Performance-Analysis

Dataset URL: https://archive.ics.uci.edu/static/public/320/

Reproducibility note: Include the notebook, README.md, requirements.txt, relevant visualization images, evaluation CSV files, and any necessary saved model/preprocessor artifacts. Avoid including sensitive information or unneeded temporary files.